GraphRAG for Competitive Programming

A Self-Correcting Hybrid Knowledge Graph & Dense Vector Retrieval System

Author: *N Bhargava | B.Tech DSAI, IIIT Bangalore* | ***Project:*** *rag_project*

1. Executive Overview & Problem Statement

Standard Retrieval-Augmented Generation (RAG) architectures rely primarily on dense bi-encoder vector embeddings. While suitable for generic semantic similarity, vector search breaks down severely in **algorithmic problem solving and competitive programming (CP)** due to four structural limitations:

- **Multi-Hop Dependency Blindness:** Questions like *'What must I learn before Suffix Arrays?'* demand strict topological graph traversal across prerequisite chains (e.g., Binary Search → Suffix Array). Vector search simply returns documents that mention both terms without respecting directed dependencies.
- **Structural Algorithmic Tradeoffs:** Queries comparing implementations (e.g., *'Segment Tree vs Fenwick Tree for range queries'*) require multi-attribute contrast (point updates vs range updates, constant factors, space complexity $O(n)$ vs $4n$). Vector similarity retrieves isolated paragraphs rather than structured tradeoffs.
- **Code vs Prose Syntax Disconnect:** Competitive programming editorials interleave English explanations with low-level C++ code. Vectorizing code often matches superficial language syntax (e.g., vector<int>) instead of algorithmic invariants.
- **Silent Hallucinations & Contradictions:** When multiple editorial sources claim conflicting complexities for variations of a technique, standard RAG averages them together into hallucinated statements instead of attributing claims to specific sources.

Key Insight: This project solves these challenges by fusing a **Neo4j Knowledge Graph** (for strict multi-hop dependencies and relational tradeoffs) with a **FAISS Dense Vector Store** (for fine-grained editorial prose and C++ code block search), orchestrated by an **Agentic Self-Grading Loop**.

2. System Architecture & Information Flow

The architecture unifies three primary subsystems: knowledge graph traversal, dense vector indexing, and an intelligent self-grading loop.

Component	Technology	Role & Core Function
Knowledge Graph	Neo4j 5.23 (Cypher)	Stores Pattern, Problem, CodeTemplate, and Source nodes. Executes multi-hop graph hops, prerequisite chains, and tradeoff traversals.
Vector Store	FAISS (IndexFlatL2)	Indexes 80+ prose chunks and C++ code blocks. Provides Top-K similarity search with metadata linking back to graph pattern IDs.
Embeddings	all-MiniLM-L6-v2	384-dimensional dense semantic representations with fast CPU inference and zero external API dependencies.

Ingestion Pipeline	BeautifulSoup4 + Claude / Heuristics	Scrapes Codeforces/LeetCode editorials, isolates C++ code blocks, extracts structured entities, and merges them into Neo4j.
Agentic Router	Python / Structured Prompts	Routes incoming user queries to graph traversal, vector search, or hybrid retrieval based on linguistic intent.
Self-Grading Evaluator	LangChain / Claude / Ragas	Grades retrieved context for relevance and sufficiency; widens search radius or cleanly abstains if out-of-corpus.

3. Step-by-Step Phase Walkthrough

The project was built incrementally across distinct engineering phases. Here is how each phase was constructed, tested, and validated:

Phase 1: Knowledge Graph Infrastructure & Cypher Traversal

Goal: Establish the graph schema, configure containerized services, seed ground-truth algorithmic relationships, and implement Cypher query algorithms.

- **Infrastructure Setup:** Authored docker-compose.yml configuring Neo4j 5.23 Community Edition with APOC plugins alongside Qdrant vector database. Added healthchecks and volume persistence.
- **Schema Constraints & Indexing:** Created schema definitions in ingestion/load_graph.py using idempotent Cypher queries. Added uniqueness constraints on (p:Pattern).name, (prob:Problem).id, and (s:Source).url, plus a Neo4j Full-Text Index on pattern descriptions.
- **Algorithmic Seed Graph:** Populated 8 canonical patterns (Two Pointers, Sliding Window, Binary Search, Prefix Sum, Fenwick Tree, Segment Tree, Sparse Table, Suffix Array) with time/space complexity properties and category tags.
- **Relational Graph Wiring:** Wired directed PREREQUISITE_OF edges (e.g., Prefix Sum → Fenwick Tree), ALTERNATIVE_TO edges with explicit trade-off metadata, SOLVED_WITH edges to 5 representative problems, and HAS_TEMPLATE edges to verified C++ code blocks.
- **Cypher Retrieval Engine:** Developed retrieval/graph_retriever.py implementing structured query execution: compare_patterns() for pairwise tradeoff evaluation, get_prerequisite_chain() using recursive variable-length path traversal, and lookup_complexity() with full-text fallback.
- **Evaluation Benchmark:** Created eval/test_queries.json defining 6 golden test cases covering graph-hop, hybrid, contradiction, and out-of-corpus queries.

Phase 2: Ingestion & Structured Knowledge Extraction

Goal: Build an automated pipeline to ingest raw editorial text and C++ code from competitive programming platforms, extract structured entities, and merge them into the knowledge graph.

- **Curated Corpus:** Built data/editorials.json with 20 real-world editorials spanning Codeforces (ratings 800–1900) and LeetCode (Easy–Hard), featuring complete editorial prose and full C++ solutions.
- **Web Scraper:** Implemented ingestion/scrape.py with HTTP client headers, conservative rate limiting (2-second request intervals), and HTML-to-Markdown conversion while preserving fenced code blocks.
- **C++ Code Block Parser:** Built ingestion/parse_cpp.py to cleanly separate natural language prose from source code. Extracted function signatures (solve(), query()), identified data structures (struct Fenwick, class SegmentTree), and detected STL containers.
- **Entity Extraction:** Created ingestion/extract_entities.py supporting dual-mode execution: (1) LLM extraction via Anthropic Claude structured JSON outputs, and (2) deterministic offline rule-based heuristic extraction allowing full local execution with zero API cost.
- **Pipeline Orchestration:** Implemented ingestion/pipeline.py which coordinates the workflow: load corpus → parse C++ → extract entities → deduplicate aliases → write intermediate data/extractions.json → merge into Neo4j.

Phase 3: Vector Store & Semantic Chunk Linking

Goal: Enable semantic similarity search over prose and code blocks, with each vector chunk explicitly linked to the Knowledge Graph via pattern IDs.

- **Lightweight Vector Engine:** Adopted **FAISS (`IndexFlatL2`)** as a pure local vector engine paired with **`all-MiniLM-L6-v2`** (384 dimensions), eliminating heavy infrastructure overhead while achieving sub-millisecond retrieval latency.
- **Chunking & Metadata Linkage:** Built retrieval/vector_store.py to chunk editorials into prose paragraphs and standalone code snippets. Attached metadata including editorial_id, chunk_type, and crucially linked_pattern_ids.
- **Disk Persistence:** Implemented serializable storage saving index.faiss, metadata.pkl, and config.json under data/vector_index/.
- **Retrieval Interface:** Built retrieval/vector_retriever.py with methods for search(), search_code(), and search_prose(). Evaluated queries like *'segment tree range query'* (score 0.47) and verified that returned chunks correctly output linked_pattern_ids for hybrid graph expansion.

Roadmap: Phases 4 through 8

- **Phase 4 (Query Router & Hybrid Fusion):** Classifies queries by intent (graph-hop, code retrieval, conceptual search) and merges Top-K vector chunks with multi-hop graph subgraphs.
- **Phase 5 (Agentic Self-Grading Loop):** Evaluates context relevance and sufficiency. Retries with widened search radius if insufficient, or cleanly triggers honest abstention if out of corpus.
- **Phase 6 (Contradiction Detection):** Detects divergent algorithmic claims across sources and surfaces them with full provenance attribution.
- **Phase 7 (Evaluation Harness):** Automated benchmarking using Ragas metrics (Faithfulness, Answer Relevance, Context Precision) over eval/test_queries.json.
- **Phase 8 (Interactive UI & API):** FastAPI endpoints and Streamlit interactive web interface.

4. Verification, Test Results & Golden Queries

The project includes a formal test suite in eval/test_queries.json. Below are the verified test queries executed against the system:

Query ID & Prompt	Type	Retrieval Mechanism & Verification Result
Q1: How does a segment tree differ from a Fenwick tree for range queries?	Graph-Hop	**Verified:** Traverses (Segment Tree)-[:ALTERNATIVE_TO]-(Fenwick Tree). Returns tradeoff: Fenwick has lower constant factor & O(n) memory; Segment Tree supports range updates with lazy propagation. Pulls both C++ templates.
Q2: What do I need to know before learning suffix arrays?	Graph-Hop	**Verified:** Recursive Cypher traversal [:PREREQUISITE_OF*1..5] returns topological ancestor chain: Binary Search → Suffix Array.
Q3: Give me a C++ template for two-pointers on a sorted array	Hybrid	**Verified:** FAISS vector search identifies Two Pointers editorial chunk → bridges to graph node Two Pointers → retrieves verified C++ template with explanation.
Q4: What's the time complexity of a sparse table?	Property	**Verified:** Direct graph property lookup on Pattern {name: 'Sparse Table'} returns O(n log n) build, O(1) query time.
Q5: Is a segment tree O(log n) or O(n log n) per query?	Contradiction	**Phase 6 Target:** Multi-source retrieval detects conflicting claims between basic segment tree O(log n) and dynamic persistent tree with walk O(log^2 n), presenting both with source links.
Q6: Explain Aho-Corasick automaton for multi-pattern matching	Abstention	**Phase 5 Target:** Detects term absent from both Neo4j and FAISS. Rejects hallucination and outputs: *'I don't have enough grounded information in the corpus'*.

5. Execution Guide & Reproduction Commands

To inspect, build, and test this project from scratch, run the following commands:

```
# 1. Environment & Dependencies
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# 2. Start Graph Database & Seed Data (Phase 1)
docker compose up -d
python -m ingestion.load_graph
python -m retrieval.graph_retriever

# 3. Run Ingestion Pipeline (Phase 2)
python -m ingestion.pipeline --mode rules --dry-run

# 4. Build Vector Index & Run Similarity Search (Phase 3)
python -m retrieval.vector_store
python -m retrieval.vector_retriever
```

6. Key Design Takeaways for AI Agents

- **Dual-Mode Fallback:** In competitive programming tasks, external APIs (like LLMs or cloud endpoints) may suffer rate limits. Ingestion and retrieval must always provide deterministic, offline rule-based fallbacks.
- **The Graph-Vector Bridge:** Dense vectors identify the semantic neighborhood, while graph nodes and edges provide structural precision. Never store vector chunks without their corresponding graph entity IDs.
- **Honest Abstention:** The highest-leverage trait of an agentic RAG system in algorithmic domains is the refusal to hallucinate non-existent algorithms or invalid C++ code.